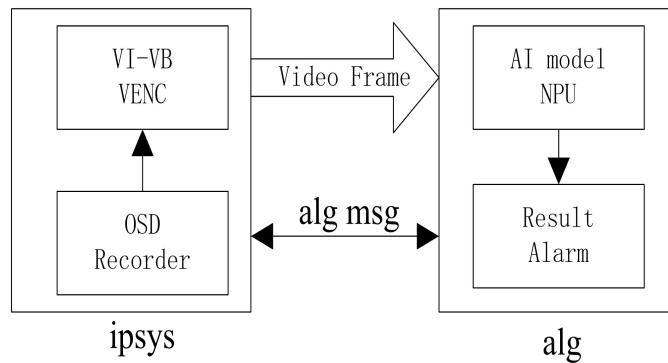


Algorithm Code Introduction

1. Overview

The alg directory stores all the codes related to the AI algorithm, which can collect the required algorithms according to the product function definition to generate the alg algorithm process to run. The alg process communicates with the ipsys process through shared messages and shared memory. The image frames are obtained from the ipsys process to perform algorithm calculations and the calculation results are notified to the ipsys process for pedestrian related alarm event processing. As shown below:



Note: alg depends on ipsys, so when debugging alg program, you need to run ipsys first and then run alg, and if you kill alg, you need to restart ipsys.

2. Code structure

```
HD900
|-- src/
|   |-- alg
|   |   |-- include          # Algorithm common header file
|   |   |   |-- alglib.h    # Pedestrian detection algorithm model library header file
|   |   |   |-- pd          # Pedestrian detection algorithm module
|   |   |   |-- dmm         # DMS detection algorithm module
|   |   |   |-- demo
|   |   |   |-- media       # Media related to using demo
|   |   |   |-- rknpu      # The RK NPU.rknn network model is loaded to run the demo
|   |   |   |-- main       # Use demo, combined with live media streaming and NPU
|   |   |-- alg.cpp         # alg process entry function
|-- build/
|   |-- ada32v2
|   |   |-- root/
|   |   |   |-- pds_normal.rknn # Pedestrian detection algorithm model
|   |   |   |-- usr/
|   |   |   |-- lib
```

```
| | | | | -- libpds_normal_rv1126.so # Pedestrian detection algorithm
model library
```

As shown above, the normally published device automatically runs an algorithm process with the file name alg, which is run by the alg.cpp and pd.cpp source files that compile the generated alg executables. When the alg runs, the dependent libpds_normal_rv1126.so dynamic library is loaded, and the pds_normal.rknn algorithm model is initialized. Then, the alg internal threads constantly acquire video frames in the ipsys for pedestrian detection operation. You have to do algorithmic secondary development, actually develop your process file instead of the existing alg file.

3. Procedure

Currently the media, rknpu, main is made available in the src / alg/demo directory. They respectively compile the resulting execution file names: media_demo, rknn_yolov5_demo, main_demo. These demo processes are parallel to alg processes, and they can only run one of them throughout the system run. These processes are run and depend on the ipsys processes, and they all need to restart the ipsys process before executing them. Also, the wtd.sh process is a daemon, please kill the wtd.sh process before running the debugger manually. for instance:

Kill the process first:

```
killall -9 wtd.sh
```

```
killall -9 ipsys
```

```
killall -9 alg
```

Perform the following process in the order:

```
/root/ipsys &
```

```
./media_demo
```

Note: the default ipsys process gives the image resolution and format to alg is: (PDS: 608x352 RGB888, DMS: 1280x720 NV12); if your algorithm needs a different resolution you can increase the parameters to specify the required resolution when running the ipsys process, for example:

```
/root/ipsys --algWidth=832 --algHeight=480 --algImgType=1 &
```

(There, the algImgType value range: 0:RGB888, 1:BGR888, 2:NV12)

The features of the main demo demonstrations currently provided are described below.

The media: 1) demonstrate how to get real-time camera images; 2) how to draw on the real-time output screen; 3) how to play the alarm sound; 4) how to send CAN interface data.

The rknpu: Demonstrate how to use the yolov5s_relu_rv1109_rv1126_out_opt.rknn algorithm model to identify the input image and output the resulting image file.

The main: demonstrate how to get real-time images from the camera and make the yolov5s_relu_rv1109_rv1126_out_opt.rknn algorithm model to target recognition and output the results to the output images.(The demo will be the best reference you use to integrate design algorithm processes)

4. APIs

Refer to the code under each demo directory.

HD900/src/alg/demo/media

HD900/src/alg/demo/rknpu

HD900/src/alg/demo/main